CMPUT 301 Lab 2

OOP Principles

Lab 2 Overview

- Review Kotlin OOP ideas used in Android
- Understand the basics of an Android screen
- ListView Example and Exercise Demo

OOP Review

- Android screens are usually represented by an activity, such as MainActivity
- Android provides parent classes, such as AppCompatActivity
- You inherit Android behaviour and override parts of it
- You store and protect app data using properties and functions

```
class MainActivity : AppCompatActivity() {
    // Android screen behaviour comes from AppCompatActivity
}
```

Inheritance

- Inheritance means one class gets behaviour from another class
- Superclass (parent)
  - AppCompatActivity is the Android class being inherited from
- Subclass (child)
  - MainActivity is our app's screen class
- The subclass will inherit all fields and methods
  (basic constructor, attributes, and all user-defined functions)

Inheritance

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContentView(R.layout.activity_main)
```

- MainActivity inherits from AppCompatActivity
  - onCreate runs when the screen is created
  - Super.onCreate(…) calls the parent class
- This lets Android perform its required setup
- Then we add our app-specific setup, such as loading the layout

Polymorphism

- **Polymorphism** means you can call the same action on different objects, and each one does it in its own way
- Animal is the **superclass** (parent class)
- Dog is the **subclass** (child class)
- Dog overrides speak() to provide its own behaviour
- The variable type is Animal, but the actual object is Dog
- Kotlin calls Dog's version of speak() at runtime

```kotlin
open class Animal {  2 Usages
    1 Override
    open fun speak() {  1 Usage
        println("some noise")
    }
}

open class Dog : Animal() {  1 Usage
    override fun speak() {
        println("woof")
    }
}

fun main() {
    val animal: Animal = Dog()
    animal.speak() // woof
}
```

Encapsulation

- When do I use private (or protected) variables?
  - Whenever you want to hide a variable from other classes
  - Whenever that data should not be known or should not be accessible by other classes
- Keep mutable data private
  - Outside code should not change it directly
- Use functions to update app state instead of modifying properties from anywhere in the app
- Encapsulation helps keep the displayed UI consistent with the app data

```
private val cities = mutableListOf("Edmonton", "Calgary", "Vancouver")
private lateinit var adapter: ArrayAdapter<String>  1 Usage

private fun addCity(city: String) {
    cities.add(city)
    adapter.notifyDataSetChanged()
```

ListView Example and Exercise (demo)